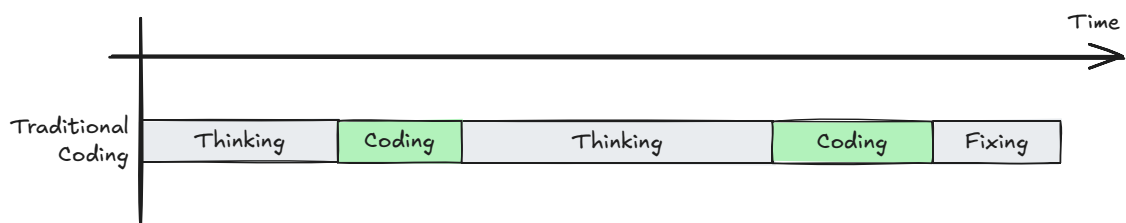


The AI coding trap | Chris Loy

10–13 minutes

If you ever watch someone “coding”, you might see them spending far more time staring into space than typing on their keyboard. No, they (probably) aren’t slacking off. Software development is fundamentally a practice of problem-solving, and so, as with solving a tricky crossword, most of the work is done in your head.

In the software development lifecycle, coding is the letters filled into the crossword, only a small amount of effort compared to all the head scratching and scribbled notes. The real work usually happens alongside coding, as the developer learns the domain, narrows down requirements, maps out relevant abstractions, considers side effects, tests features incrementally, and finally squashes bugs that survived this rigorous process. It looks something like this:

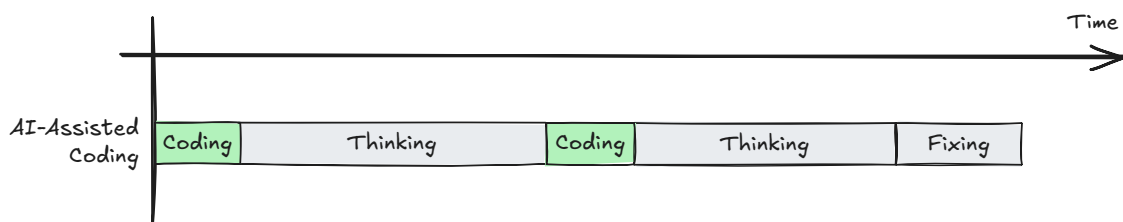


But with AI-driven coding, things play out very differently.

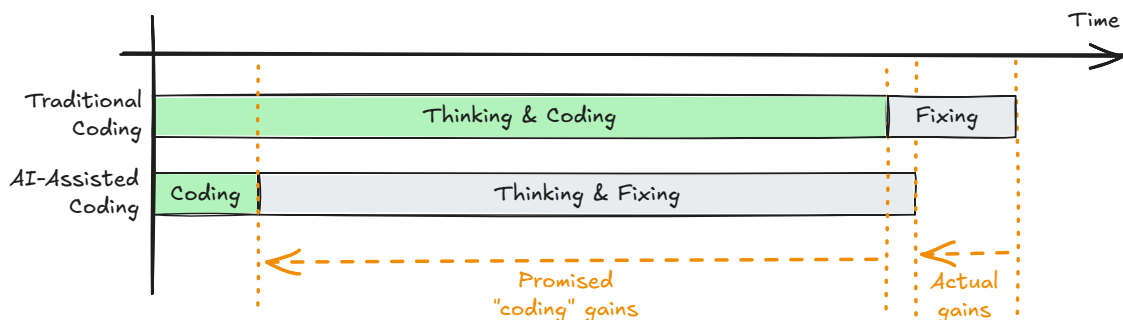
“Code first, ask questions later”

AI coding agents such as [Claude Code](#) are making it

astonishingly fast to write code in isolation. But most software lives within complex systems, and since LLMs can't yet hold the full context of an application in memory at once, human review, testing, and integration needs will remain. And that is a lot harder when the code has been written without the human thinking about it. As a result, for complex software, much of the time will be spent on post hoc understanding of what code the AI has written.



This is the root of the difference between marketing copy that boasts of the paradigm shifting speed of **writing code** with AI (often framed as “[10X faster](#)”), and the marginal productivity gains in **delivering working software** seen in the wild (usually [closer to 10%](#)).



An even more dispiriting upshot of this is that, as developers, we spend an ever greater proportion of our time merely fixing up the output of these wondrous [babbling machines](#). While the LLMs get to blast through all the fun, easy work at lightning speed, we are then left with all the thankless tasks: testing to ensure existing functionality isn't broken, clearing out duplicated code, writing documentation, handling deployment and

infrastructure, etc. Very little time is actually dedicated to the thing that developers actually love doing: coding.

Fortunately, help is at hand. While LLMs are shaking up how software development is performed, this issue in itself is not actually new. In fact, it is merely a stark example of an age-old problem, which I call:

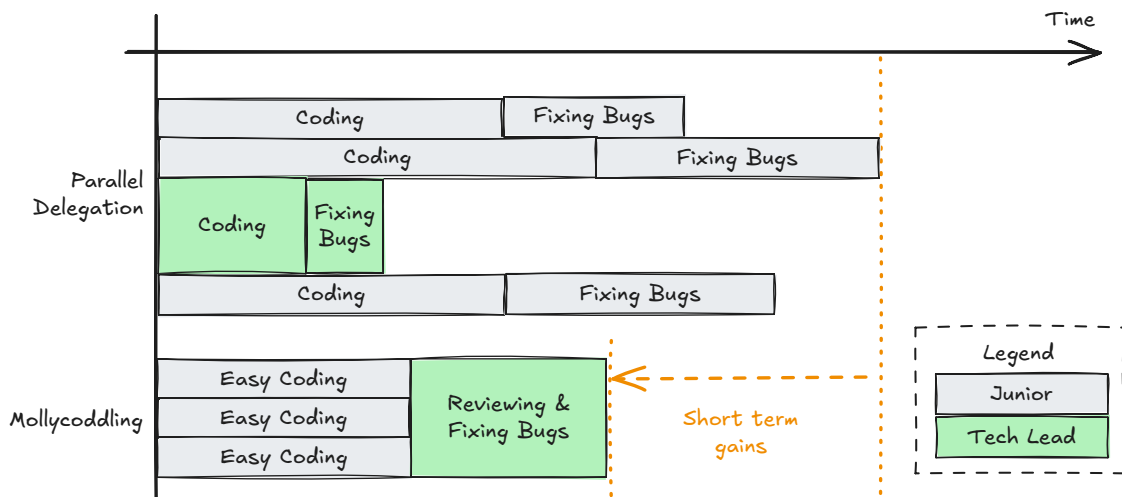
The tech lead's dilemma

As engineers progress in their careers, they will eventually step into the role of **tech lead**. They might be **managing** a team, or they could be a **principal engineer**, driving technical delivery without the people management. In either case, they are responsible for the team's technical delivery. They are also usually the most experienced developer in the team: either in their career, in the specialised domain of the team, or in both.

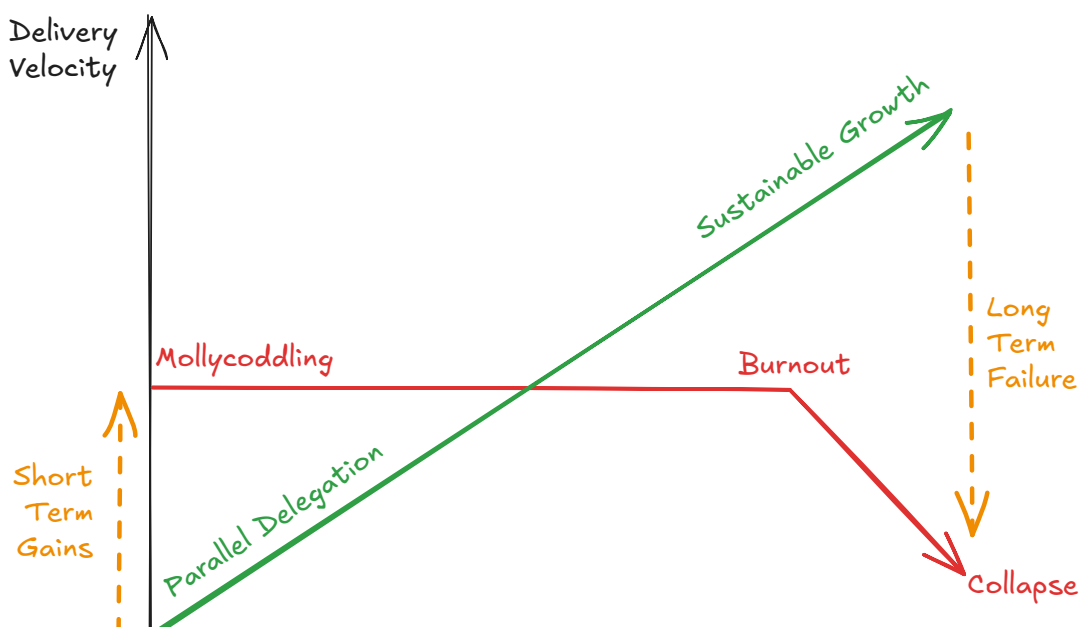
Software delivery is a team effort, but one in which experience can have a highly imbalancing effect on individual contribution [velocity](#). As such, when the tech lead's primary job is to maximise delivery, they will often face an internal conflict between two ways to deliver software:

- **Fair delegation** across the team, maximising learning and ownership opportunities for junior team members, but allowing delivery to be bottlenecked by the speed of the least productive team members.
- **Mollycoddling** the team, by delegating only the easy or non-critical work to juniors, and keeping the hardest work for themselves, as the person on the team most capable of delivering at speed.

Unfortunately, while we shall see that mollycoddling is extremely harmful to long-term team health, it is also often a very effective way to accelerate delivery. The higher bandwidth of the tech lead is often most efficiently deployed by eating up all the hardest work:



As such, I have seen this pattern repeated time and again over the course of my career. And, of course, it comes at a cost. Siloing of experience in the tech lead makes the team brittle, it makes support harder, and it places ever greater pressure on the tech lead as a single point of failure. What follows next is predictable: burnout, departure, and ensuing crisis as the team struggles to survive without the one person who really knows how everything works.





As is usually the case, the solution lies in a third way that avoids these two extremes and balances delivery with team growth. We might frame it as something like:

Implement team practices that allow engineers to deliver working code within a framework that minimises rework, maximises effective collaboration, and promotes personal growth and learning.

When I was CTO of Datasine, we enshrined this attitude in a simple tech team motto:

Learn. Deliver. Have fun.

Good tech leads expose their engineers to work at the limit of their capabilities, using processes and practices that minimise delivery risk while also enabling each team member to grow their skills, knowledge, and domain expertise. This is, in fact, the essence of good technical leadership.

There are many ways to accomplish it, from strict codified frameworks such as the [Extreme Programming rules](#), through to looser sets of principles which we might broadly refer to as “best practices”:

- Code reviews
- Incremental delivery
- [Modular design](#)
- Test-driven development
- Pair programming
- Quality documentation

- Continuous integration

So, for experienced engineers today, an urgent question is: how can we translate these practices into a world of AI-driven coding?

LLMs are lightning fast junior engineers

In 2025, many engineers are finding themselves for the first time in a position familiar to every tech lead: overseeing a brilliant but unpredictable junior engineer. Harnessing and controlling such talent, in a way that benefits effective team collaboration, is one of the primary challenges of engineering leadership. But AI coding agents need different management to junior engineers, because the nature of their productivity and growth is fundamentally different.

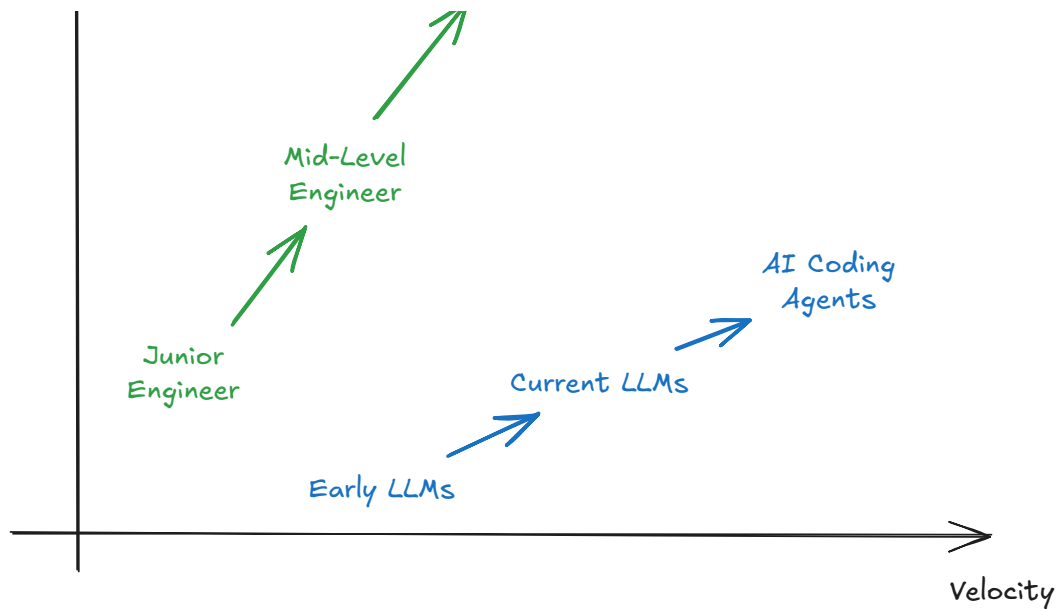
As software engineers gain experience, we tend to improve our productivity in multiple ways at the same time: writing more robust code, using better abstractions, spending less time writing and fixing bugs, understanding more complex architectures, covering edge cases more effectively, spotting repeated patterns earlier, etc. Engineering is a rich and complex discipline with many avenues for specialisation, but for simplicity we might group these dimensions into two broad themes:

- **Quality:** ability to deliver more complex, more performant, more maintainable code
- **Velocity:** ability to develop working, bug-free code in a shorter space of time

Over time, good engineers will improve in both axes.

Quality ↑

Senior
Engineer ↗



Early LLMs were fast to write code, but time spent fixing bugs and removing hallucinations meant they were slow to complete bug-free code. Over time, smarter LLMs and better use of context engineering and tools have meant that modern AI coding agents are much better at “one shot” writing of code. The current generation of commercially available agents can be incredibly fast at producing working code for problems that would challenge some mid-level engineers, though they cannot yet match the expertise of senior engineers:

So we can think of the current generation of AI coding agents as junior engineers, albeit with two fundamental differences:

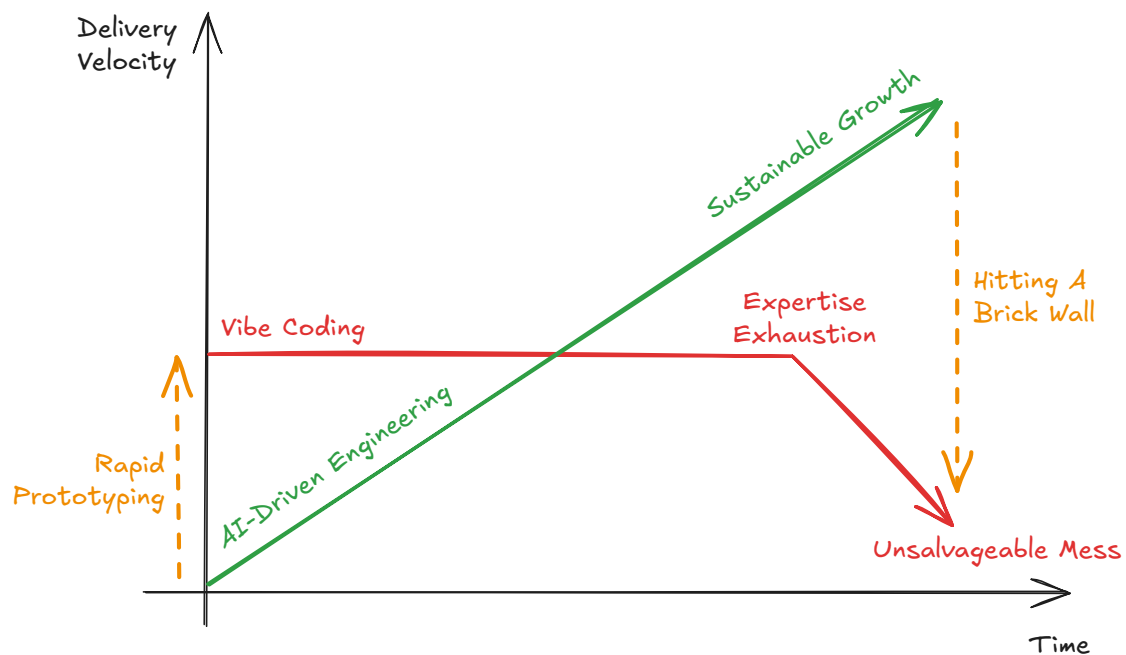
1. LLMs deliver code much, much faster than junior engineers, constrained neither by thinking nor writing time;
2. LLMs have no true capacity to learn, and instead only improve through more effective [context engineering](#) or the arrival of new foundation models.

As with junior engineering talent, there are broadly two

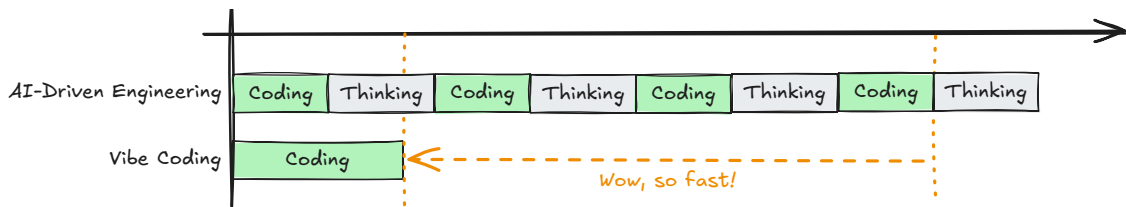
ways that you can deploy them, depending on whether your focus is long-term or short-term:

- **AI-driven engineering:** employing best practices, foregrounding human understanding of the code, moving slowly to make development sustainable.
- **Vibe coding:** throwing caution to the wind and implementing at speed, sacrificing understanding for delivery velocity, hitting an eventual wall of unsalvageable, messy code.

As might be expected, the long-term trajectories of choosing between these two approaches follow much the same pattern as choosing between parallel delegation and mollycoddling of a junior team:



This is why the [vibe coding](#) approach is great for tiny projects or throwaway prototypes: applications of sufficient simplicity can be delivered without the need for any human thinking at all. By limiting the complexity of our projects and leaning into the capabilities of the tools, we can deliver end-to-end working software in no time at all.



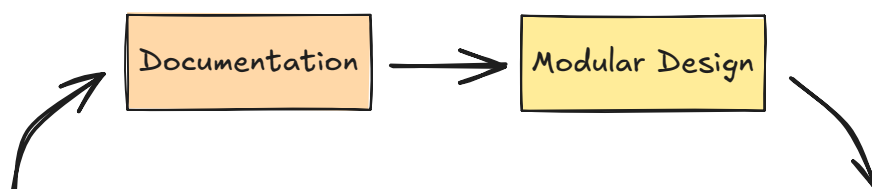
But you *will* hit a wall of complexity that AI is incapable of scaling alone.

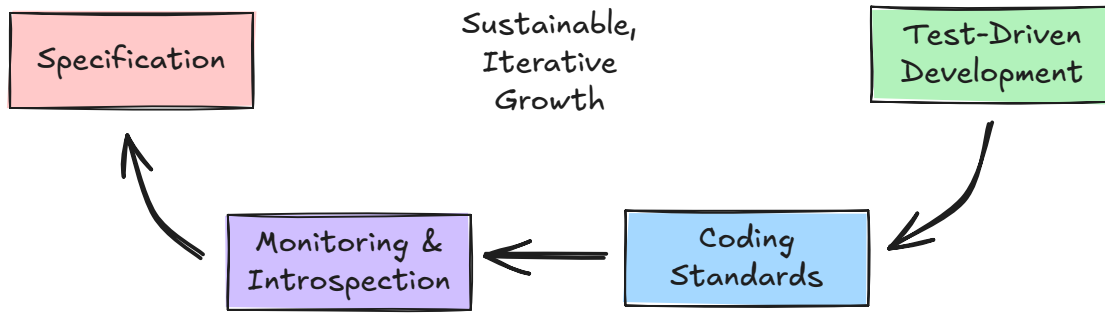
Building prototypes is now easier than ever. But if we want to effectively use LLMs to accelerate delivery of real, complex, secure, working software, and to realise more than marginal efficiency gains, we need to write a new playbook of engineering practices tailored to maximise collaboration between engineering teams that include both humans and LLMs.

How to avoid the AI coding trap

AI coding agents are dazzlingly productive, but lack in-depth knowledge of your business, codebase, or roadmap. Left unchecked, they will happily churn out thousands of lines of code with no heed paid to design, consistency, or maintainability. The job of the engineer, then, is to act as a tech lead to these hotshots: to provide the structure, standards, and processes that convert raw speed into sustainable delivery.

We need a new playbook for how to deliver working software efficiently, and we can look to the past to learn how to do that. By treating LLMs as **lightning-fast junior engineers**, we can lean on best practices from the software development lifecycle to build systems that scale.





Just as tech leads don't just write code but set practices for the team, engineers now need to set practices for AI agents. That means bringing AI into every stage of the lifecycle:

Specification: exploring, analysing, and refining feature specifications to cover edge cases and narrow focus.

Documentation: generating and reviewing documentation up front to provide reusable guardrails and lasting evidence.

Modular Design: scaffolding modular architectures to control context scope and maximise comprehension.

Test-Driven Development: generating extensive test cases prior to implementation to guide implementation and prevent regression.

Coding Standards: applying house styles and best practice when generating code, through context engineering.

Monitoring & Introspection: analysing logs and extracting insights faster than any human ever could.

By understanding that delivering software is so much more than just writing code, we can avoid the AI coding trap and instead hugely amplify our ability to deliver working, scalable software.